MINOR PROJECT PRESENTATION | B.TECH CSE

MEDUSA AI

An Advanced Multimodal Conversational AI System with Dual-Engine Inference, Live Web Reasoning, and Distributed Telegram & CLI Deployment

PROJECT TEAM MEMBERS

- Wasim Akram (Reg. No: 2301348256)
- Satyajit Samantaray (Reg. No: 2301348203)
- Susanta Kumar Parida (Reg. No: 2421348051)
- Somanath Sahoo (Reg. No: 2421348050)
- Walter Pradhan (Reg. No: 2301348255)

ACADEMIC INSTITUTION

Guide: Prof. Achinta Kumar Palit

Dept: Computer Science & Engineering

College: GIET, Bhubaneswar

Affiliation: BPUT, Odisha (2023 - 2027)

EXECUTIVE SUMMARY

1. Introduction & Project Overview

🤖 What is Medusa AI?

An enterprise-grade, privacy-respecting conversational artificial intelligence system.

- Bridges high-velocity cloud inference with private local edge computation.
- Features real-time live web search augmentation.
- Eliminates dependency on expensive, slow web-based GUI wrappers.

🌐 Dual-Access Deployment

Distributed across both mobile messaging and developer workstations.

- Telegram Bot (@aimedusaroobot): Public access worldwide for 1-on-1 chats and groups.
- Terminal CLI (medusaai): Pip-installable global package for developers with zero GUI overhead.

⚡ Core Innovations

Modern breakthroughs integrated into a single cohesive architecture.

- Dual Engine: Cloud Groq LPU + Local NVIDIA RTX GPU llama-server.
- Character Recognition: Names anime, gaming, and movie figures accurately.
- Strict Privacy: Exclusively identifies as Medusa AI with zero model leaks.

THE CHALLENGE

2. Problem Statement & Motivation

🔒 Privacy & Vendor Lock-In

KEY ISSUE: CLOUD-ONLY CENTRALIZATION

Most modern chatbots require transmitting private conversations and confidential files to proprietary third-party servers. Users lose ownership of data, and service terminates when offline.

🗓️ Knowledge Horizon Staleness

KEY ISSUE: OUTDATED INFORMATION

Pre-trained LLMs have static knowledge cutoffs. They cannot answer questions regarding breaking news, live sports, today's stock valuations, or recent events without expensive vector databases.

🖼️ Multimodal Blindspots

KEY ISSUE: SUPERFICIAL VISION

When shown an image, standard bots provide superficial artistic commentary (colors, lighting) rather than recognizing specific characters (anime/gaming/celebrity) or handling photo follow-ups.

💻 Heavy GUI Frameworks

KEY ISSUE: TERMINAL NEGLECT

Electron-based desktop wrappers consume 1-2 GB of RAM just to render chat bubbles. Developers working inside terminal command-lines lack lightweight, installable AI tools.

DESIGN GOALS

3. Key System Objectives

01. Dual-Engine Hybrid Inference

Seamlessly toggle between ultra-fast cloud inference (Groq LPUs) and 100% offline edge neural inference (NVIDIA RTX GPU with GGUF quantization).

02. Live Web Search Augmentation

Dynamically detect real-time informational intent and fetch up-to-the-minute web snippets via DuckDuckGo without requiring persistent vector databases.

03. Deep Multimodal Intelligence

Accurately recognize people, anime/game characters, and landmarks from photos; parse multi-page PDFs, inspect ZIP software archives, and transcribe audio voice clips.

04. Reliable Communication & Formatting

Provide a robust Telegram Bot with safe Markdown-to-HTML conversion that eliminates raw asterisks (***) and respects group privacy through mention filtering.

05. Global CLI Software Packaging

Distribute the terminal client as an independent Python package (`medusaai`) installable with pip, featuring ASCII Figlet banners and persistent API key management.

06. Complete Model Privacy & Secrecy

Strictly protect model identity so that backend architectures, weights, and providers are never revealed to end users, consistently maintaining the Medusa AI persona.

SYSTEM DESIGN

4. System Architecture & Dual-Engine Workflow

⚡ DEFAULT ENGINE (Cloud Accelerated)

- Core Technology: Groq Language Processing Unit (LPU) API
- Active Model: Qwen / LLaMA high-capacity reasoning model
- Token Velocity: Extremely fast (>300 tokens/second)
- Live Web Search: Integrated via DuckDuckGo Real-Time Search
- Activation Command: /default (Primary mode for all chats)
- Best Suited For: Complex reasoning, long multi-turn dialogues, current news, live facts, and instant responses.

MEDUSA ENGINE (Local Neural Model)

- Core Technology: llama-server C++ Subprocess on Port 8088
- Active Model: medusa-1.8-flash.gguf (Q4_K Medium Quantization)
- Hardware Offload: 100% layer offloading to NVIDIA RTX 2050 GPU
- Data Sovereignty: 100% Private, zero cloud telemetry or logs
- Activation Command: /medusa (Secondary local mode)
- Offline Failover: If server is stopped, bot gracefully warns: '⚠️ Models are offline now. Please switch with /default.'

ZERO-VECTOR RAG

5. Real-Time Web Search Augmentation

💡 Dynamic Zero-Vector Web Retrieval Pipeline

Unlike traditional RAG systems that require heavy embedding databases, Medusa AI employs an on-demand, heuristic-driven search pipeline. Incoming user prompts are parsed by regex intent classifiers; if real-time information is required, the system queries the live internet and synthesizes facts directly into the prompt context.

Step 1: Heuristic Detection

Compiled regex matches temporal triggers: 'today', 'latest', 'current', 'news', 'price', 'weather', 'who won'.

Step 2: Query Normalization

Conversational fluff ('please search for', 'tell me about') is stripped to form clean, high-precision search keywords.

Step 3: DuckDuckGo Live Query

The engine fetches top 4 factual snippets, titles, and source URLs asynchronously in an executor threadpool.

Step 4: Context Injection

Extracted web snippets are injected into an ephemeral system prompt, allowing the LLM to generate cited, up-to-date facts.

VISUAL INTELLIGENCE

6. Multimodal Vision & Character Recognition

🎯 Character Recognition Breakthrough

Naming the subject instead of generic art critique.

• Engineered specialized prompts instructing Gemini to prioritize identifying characters from anime, manga, video games, comics, movies, and pop culture.

• Example: An anime girl with braided orange hair, yellow concentric eyes, and a halo is immediately recognized as 'Makima from Chainsaw Man', complete with background lore.

💬 Contextual Photo Replies

Seamless multi-turn image follow-ups.

• Detects when a user replies to any photo message with 'who is she?' or 'what's her name?'.

• Caches the last uploaded photo (`self.last_photo`) per chat, enabling natural follow-up questions without re-uploading imagery.

📄 Document & Archive Analysis

Deep multi-page extraction & code review.

• PDF Documents: Reads multi-page research papers and reports via PyMuPDF (fitz) up to 50 pages.

• ZIP Archives: Unpacks file manifests, inspects code files (.py, .js, .json, .md), and generates structural architectural summaries.

FRONTEND DEPLOYMENT

7. Telegram Bot Implementation & UX

⚡ python-telegram-bot v22 Async

Built on a non-blocking asynchronous event loop with continuous typing actions (`ChatAction.TYPING`) to provide immediate visual feedback while inference executes.

🎛️ Interactive Inline Keyboards

Interactive buttons embedded into `/start` allow users to switch engines, trigger quick help, or clear memory with a single tap, bypassing manual command typing.

👥 Intelligent Group Privacy

In Telegram group chats, the bot stays silent during standard chatter. It activates exclusively when directly @mentioned or replied to, keeping groups clean.

🛡️ Deterministic HTML Formatter

Converts markdown (`**bold**`, `***bold-italic***`, headers, code blocks) into Telegram HTML. Eliminates ugly unrendered asterisks (***) with safe plain-text fallback.

✂️ 4096-Character Message Chunking

Telegram limits messages to 4,096 characters. Medusa AI automatically splits long essays and code listings by paragraphs, delivering complete replies seamlessly.

🧠 Multi-Turn Memory Sliding Window

Maintains conversational context per chat up to 12 turns, allowing users to ask natural follow-up questions while preventing token budget exhaustion.

DEVELOPER DISTRIBUTION

8. Standalone CLI Tool (medusaai)

📦 Independent Python Package

- Package Name: medusaai (Version 1.0.0)
- Installation: pip install medusaai (or pip install .)
- Console Commands: medusaai or simply medusa
- Directory: Completely isolated in C:\...\medusaai
- Build Artifacts: Built .whl and .tar.gz in dist/ ready for PyPI
- Credential Storage: Stored once in ~/.medusaai/config.json with zero repeat prompts.

💻 Terminal REPL Interface

- PyFiglet ASCII Banner: Renders bold 'Medusa' branding in crimson red.
- Live Timestamp & Author: Displays centered Date, Time, and 'Author: Tsukiziro'.
- Mode Indicator: Displays active engine status (MODE: DEFAULT / MEDUSA).
- Real-Time Web Triggers: Automatically searches the web for current questions.
- Colorama Styling: Highlighted terminal text without raw asterisks (***).
- Commands: /default, /medusa, /key <api_key>, /clear, /exit.

EMPIRICAL RESULTS

9. Performance Benchmarks & Testing

CLOUD GENERATION SPEED

310 tokens/sec

Groq LPU accelerated inference

LOCAL MODEL VELOCITY

27.9 tokens/sec

NVIDIA RTX 2050 (All layers GPU offloaded)

CLOUD RESPONSE LATENCY

340 ms

Sub-second Time-to-First-Token

VRAM UTILIZATION

2.28 GB

Comfortably fits within 4GB consumer VRAM

🧪 Verification & Test Case Highlights (16 / 16 PASSED)

✓ TC-01 & TC-02: Slash command registration and inline keyboard callback handling verified.
✓ TC-04 & TC-05: Local Medusa engine connection and graceful offline failover message confirmed.
✓ TC-06 & TC-07: Real-time news and live crypto price retrieval tested via DuckDuckGo scraping.
✓ TC-09 & TC-10: Anime character identification (Makima) and photo reply follow-ups verified.
✓ TC-13 & TC-14: Deterministic HTML rendering (zero asterisks) and 4096-char chunk splitting verified.
✓ TC-15 & TC-16: Model secrecy guardrails and global pip CLI execution verified.

VALIDATION

10. Key Results & Practical Demonstrations

📱 Live Telegram Bot

Active Worldwide Access

- Bot Username: @aimedusaroobot
- Responds with typing indicators in <1 second.
- Handles dual-engine switching, inline buttons, and multi-user concurrent sessions effortlessly.
- Zero formatting errors: headers, bold text, and code blocks render with native elegance.

🖼️ Visual Recognition

Accurate Character Identification

- Successfully recognized complex anime characters from raw user imagery.
- Accurately parsed PDF research papers, returning structured chapter summaries.
- Unpacked software ZIP archives, analyzing multi-file codebases in seconds.

🔒 Persona Confidentiality

Strict Model Anonymity

- Complete protection against prompt leaks: The model consistently identifies exclusively as Medusa AI.
- When asked 'what model are you?', it never exposes Groq, Gemini, Qwen, or LLaMA.
- The /model command was permanently decommissioned.

PROJECT WRAP-UP

11. Conclusion & Future Roadmap

🌟 Summary of Contributions

- Successfully engineered a hybrid edge-cloud conversational ecosystem.
- Solved the model privacy dilemma by combining local RTX GPU execution with zero cloud telemetry.
- Implemented zero-vector live web search for up-to-date factual intelligence.
- Resolved multimodal character identification breakdowns for anime and gaming imagery.
- Packaged a standalone pip library (`medusaai`) for developers worldwide.

🚀 Future Enhancements

1. Bidirectional Voice: Real-time speech-to-speech with Whisper.cpp and Edge-TTS.
2. Fully Offline Vision: Integrating quantized MiniCPM-V GGUF into llama-server.
3. Webhook Production Scaling: Deploying behind Cloudflare reverse proxies.
4. Persistent Long-Term Memory: Vector-based user conversation recall.

Thank You! Questions & Discussion Welcome.